

# QB: Chapter 1.1 — Fundamentals of Data Structures & Arrays

Unit 1 |  $5 \times 2$ -mark |  $5 \times 5$ -mark |  $4 \times 10$ -mark | **Total: 14 questions**

Lectures: L01–L06 | BT Levels: BT1, BT2, BT3, BT4 | COs: CO1, CO2, CO5

---

## Section A: 2-Mark Questions

**Q1.** [BT1 | CO1] Define the term *data structure*. Classify data structures into their two primary categories and name two examples of each.

---

**Q2.** [BT1 | CO1] State the three asymptotic notations — Big-O, Big- $\Omega$ , and Big- $\Theta$ . Write the condition that must hold simultaneously for  $f(n) = \Theta(g(n))$  to be true.

---

**Q3.** [BT2 | CO2] Describe how elements of a 1D integer array  $A[n]$  are stored in contiguous memory. Write the formula to compute the address of  $A[i]$  given base address BA and element size  $w$  bytes.

---

**Q4.** [BT2 | CO1] Explain the concept of time–space tradeoff. Give one concrete example where using additional memory reduces the execution time of an algorithm.

---

**Q5.** [BT3 | CO2] Array  $A = \{4, 11, 22, 35, 48\}$  is stored at base address  $BA = 2000$  with each element occupying 4 bytes. Calculate the memory addresses of  $A[0]$  and  $A[4]$ . Show working.

---

## Section B: 5-Mark Questions

**Q6.** [BT3 | CO2] Trace the insertion of element 55 at index 2 in the array  $[10, 20, 30, 40, 50]$  (capacity = 6, current size = 5). Show the state of the array after each rightward shift. Determine the worst-case time complexity of insertion at an arbitrary position and justify with a step count.

---

**Q7.** [BT3 | CO2] Implement deletion of an element at position  $p$  in an array of  $n$  elements. Demonstrate the algorithm by deleting the element at index 1 from the array  $[5, 15, 25, 35, 45]$ . Show each leftward shift and the array state after deletion. State time and space complexity.

---

**Q8.** [BT4 | CO1] Formally prove that  $f(n) = 3n^2 + 8n + 5$  satisfies both  $O(n^2)$  and  $\Omega(n^2)$ , and hence conclude its  $\Theta$  class. Provide explicit constants  $c_1$ ,  $c_2$ , and  $n_0$  for each bound, and verify the definition inequalities.

---

**Q9.** [BT4 | CO1] Construct a comparison table for the six common complexity classes:  $O(1)$ ,  $O(\log n)$ ,  $O(n)$ ,  $O(n \log n)$ ,  $O(n^2)$ ,  $O(2^n)$ . For each, provide: the class name, one algorithm example, and the approximate operation count for  $n = 64$ . Use this to explain why  $O(2^n)$  algorithms are impractical for  $n > 30$ .

---

**Q10.** [BT4 | CO1, CO5] Determine the asymptotic time complexity of the following two code fragments. Justify each answer by counting the number of executed iterations as a function of  $n$ .

Fragment A:

```
for i in range(n):
    for j in range(i):
        print(i, j)
```

Fragment B:

```
while n > 1:
    n = n // 2
```

---

## Section C: 10-Mark Questions

**Q11.** [BT4 | CO1]

A software engineer is analysing two candidate algorithms, P and Q, for processing a dataset of size  $n$ :

- Algorithm P performs  $T_P(n) = 4n^2 + 6n + 2$  operations.
- Algorithm Q performs  $T_Q(n) = 80n \log_2 n + 100$  operations.

**(a)** Express  $T_P(n)$  and  $T_Q(n)$  in Big- $\Theta$  notation. Justify each using the formal definition with explicit constants  $c_1$ ,  $c_2$ , and  $n_0$ .

**(b)** Determine the approximate crossover value  $n_0$  beyond which Q outperforms P. Set up the inequality  $T_Q(n) < T_P(n)$  and solve numerically.

**(c)** For a dataset of size  $n = 10,000$ , calculate the approximate operation count of each algorithm. Select the better algorithm for this dataset and explain the importance of worst-case analysis in practical algorithm selection.

---

**Q12.** [BT4 | CO1, CO2]

A 2D integer array  $A[4][5]$  is stored with base address  $BA = 5000$  and integer size = 4 bytes.

**(a)** Derive the address formula for element  $A[i][j]$  in row-major order. Using this formula, calculate the addresses of  $A[2][3]$  and  $A[3][4]$ . Show all steps.

**(b)** Re-derive the address formula for column-major order. Calculate the address of  $A[2][3]$  under column-major storage. Explain which programming languages use each ordering.

**(c)** If the same data were instead stored as a 3D integer array  $B[2][2][5]$  with  $BA = 5000$ ,

write the general row-major address formula for  $B[i][j][k]$  and compute the address of  $B[1][1][3]$ .

---

**Q13.** [BT4 | CO1, CO5] (*Experiment E1 — Search in Rotated Sorted Array, LeetCode #33*)

A sorted integer array  $A = [7, 9, 14, 17, 22, 3, 5, 6]$  has been rotated at an unknown pivot index.

(a) Explain why standard binary search fails on this array with an example showing an incorrect decision. Describe the modified binary search algorithm that correctly identifies the sorted half at each step and searches within it.

(b) Trace the modified binary search for target = 3 on the array above. At each step, record the values of `low`, `mid`, `high`, and the decision made (which half is sorted and which half is searched next).

(c) Derive the time complexity of the modified binary search and prove it is  $O(\log n)$ . Compare the number of comparisons made in this trace to what a linear scan would require. State the precondition this algorithm imposes on the input.

---

**Q14.** [BT4 | CO2]

An integer array of capacity 10,000 is used as a sliding window buffer where elements are continuously inserted at one end and removed from the other.

(a) Analyse the time complexity of inserting at the front (index 0) and deleting from the rear (last index) of a static array. Derive the exact step count in terms of  $n$  and express it in Big-O notation.

(b) Explain how a circular array with modulo arithmetic eliminates the shifting inefficiency. Write pseudocode for `circular_enqueue(item)` and `circular_dequeue()` that uses `front`, `rear`, and `size` variables. Show how wrap-around is handled.

(c) Compare array, singly linked list, and circular array for this sliding window use case across the following criteria: time complexity for insert/delete at each end, memory overhead per element, and cache locality. Recommend the optimal structure and justify.